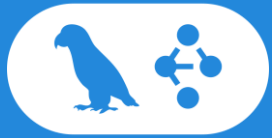


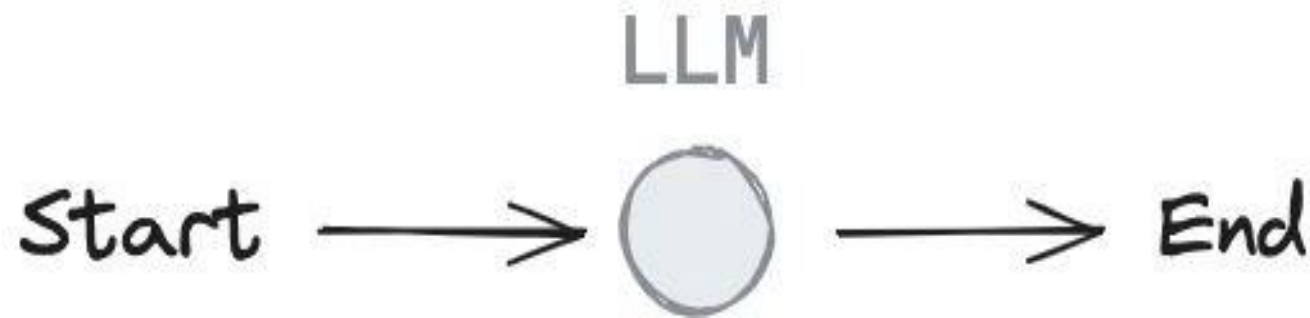
LangGraph



LangGraph, created by LangChain, is an open source AI agent framework designed to build, deploy and manage complex **Generative AI agent workflows.**



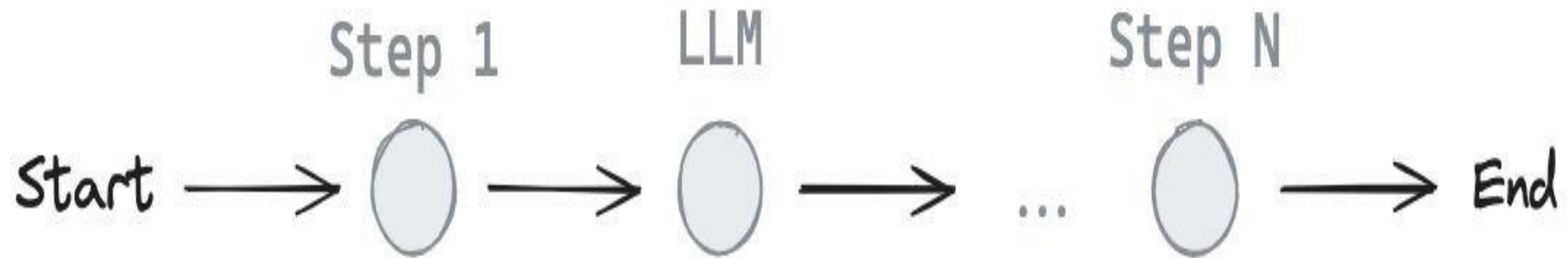
A solitary language model is fairly limited...



...e.g., access to tools, external context, multi-step workflows.



So, many LLM applications use a control flow ...

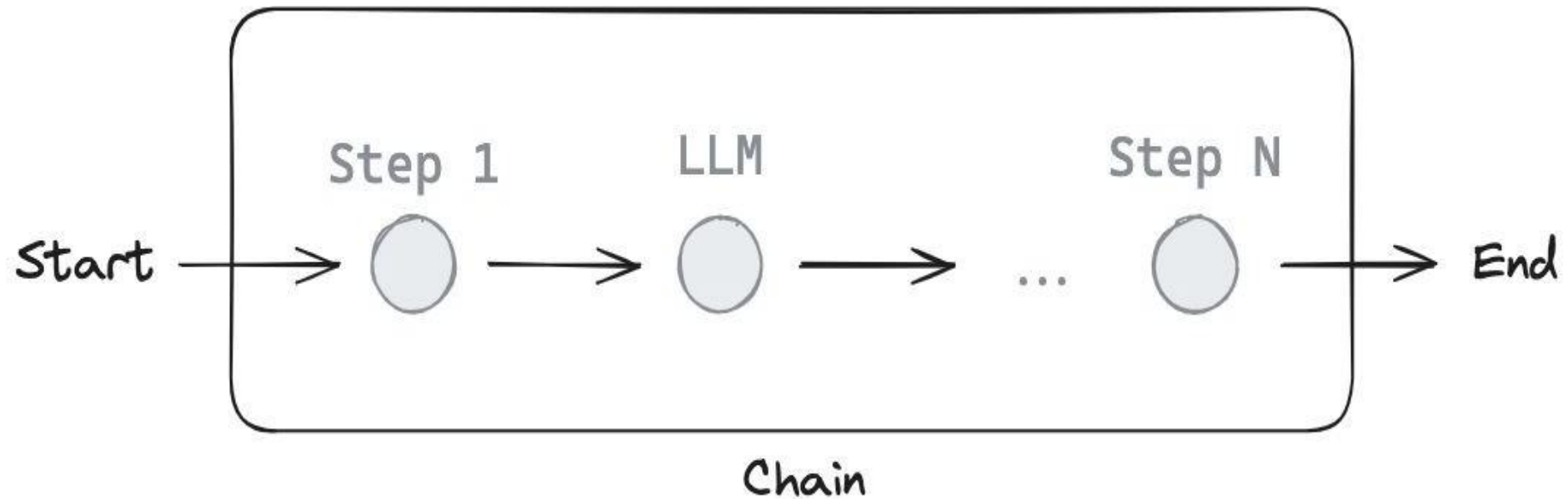


...with steps pre / post-LLM call (tool calls, retrieval, etc).

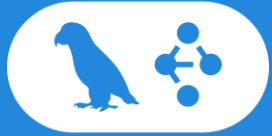
LangGraph: Chain



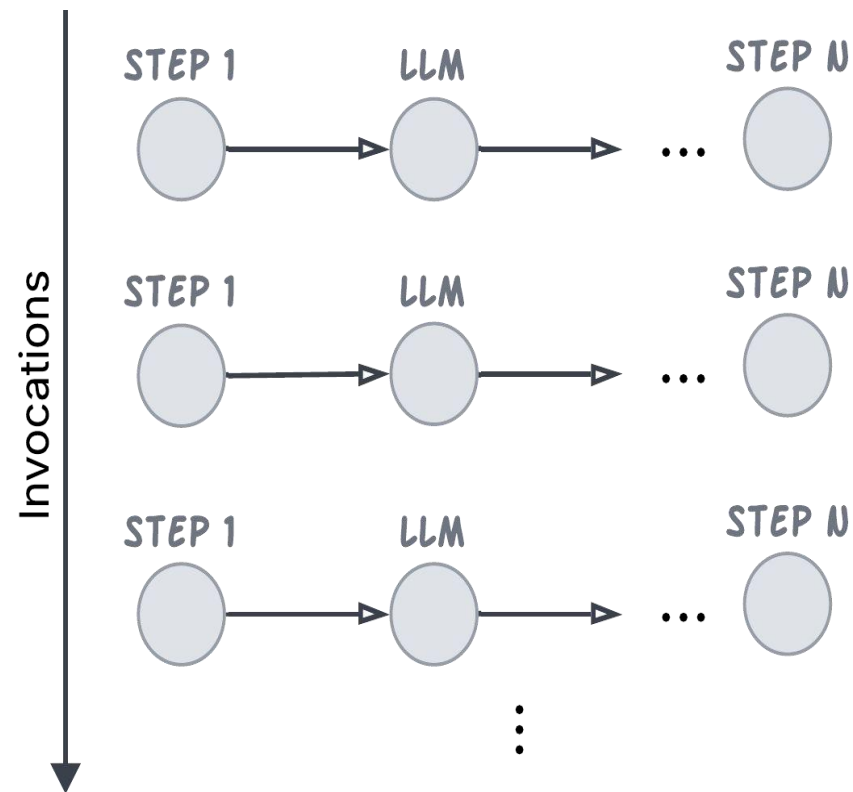
This control flow forms a “chain”



LangGraph: Chain

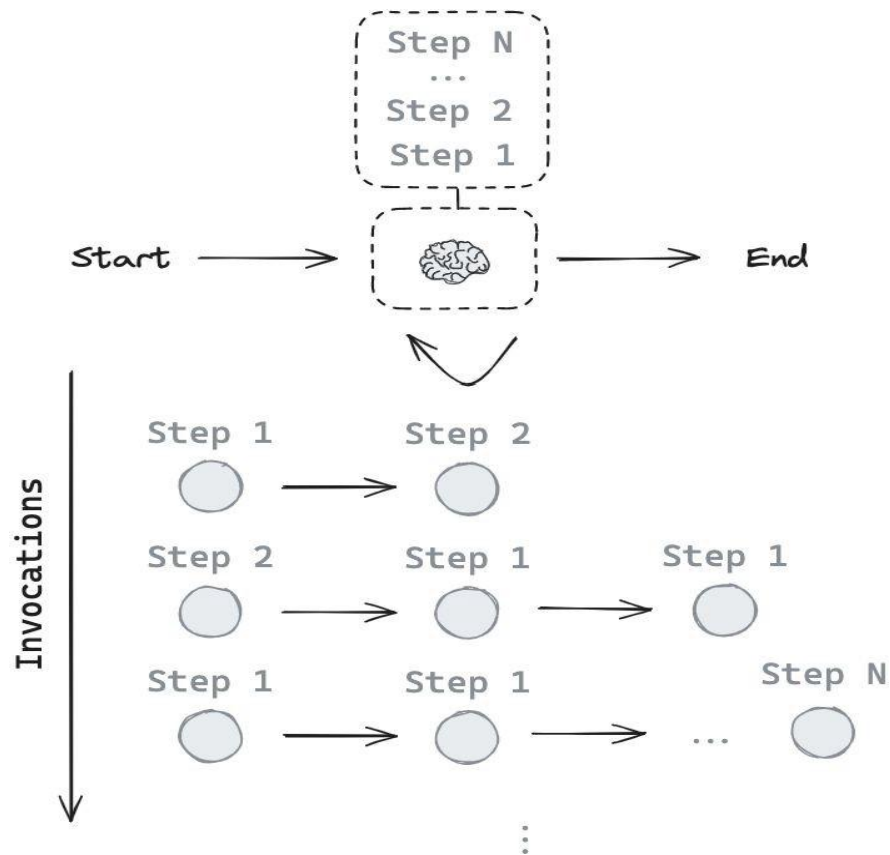


Chains are reliable! Same control flow every time

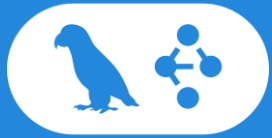




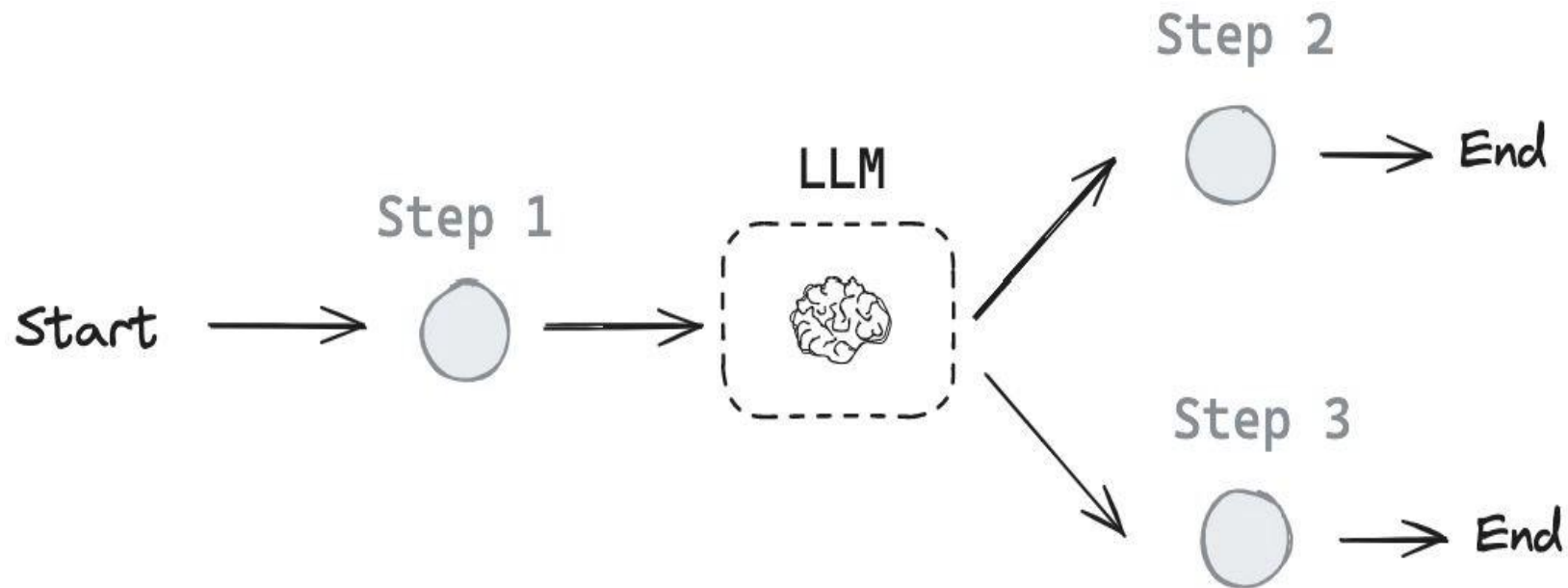
But, we want LLM systems that can pick their own control flow!



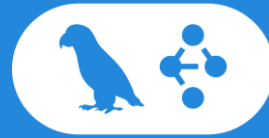
LangGraph: Agent



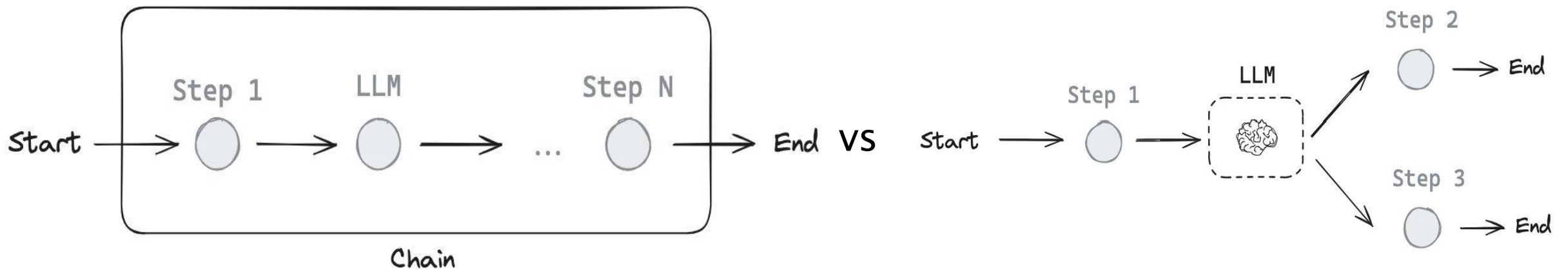
Agent $\sim$ control flow defined by an LLM

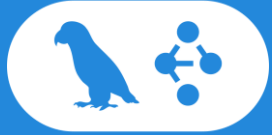


LangGraph: Agent



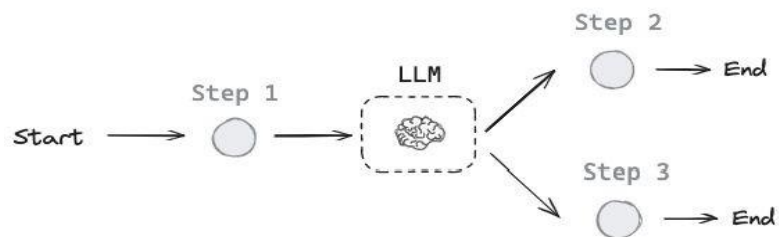
Fixed vs LLM-defined control flow



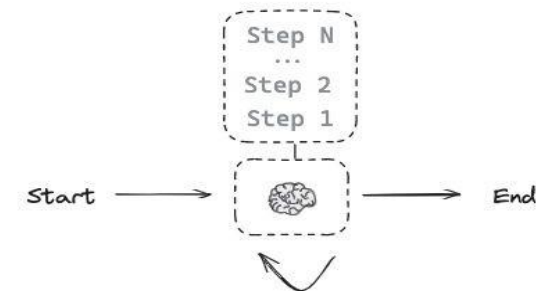


Many kinds of agents!

Router



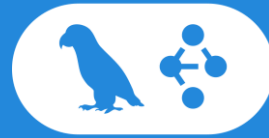
Fully Autonomous



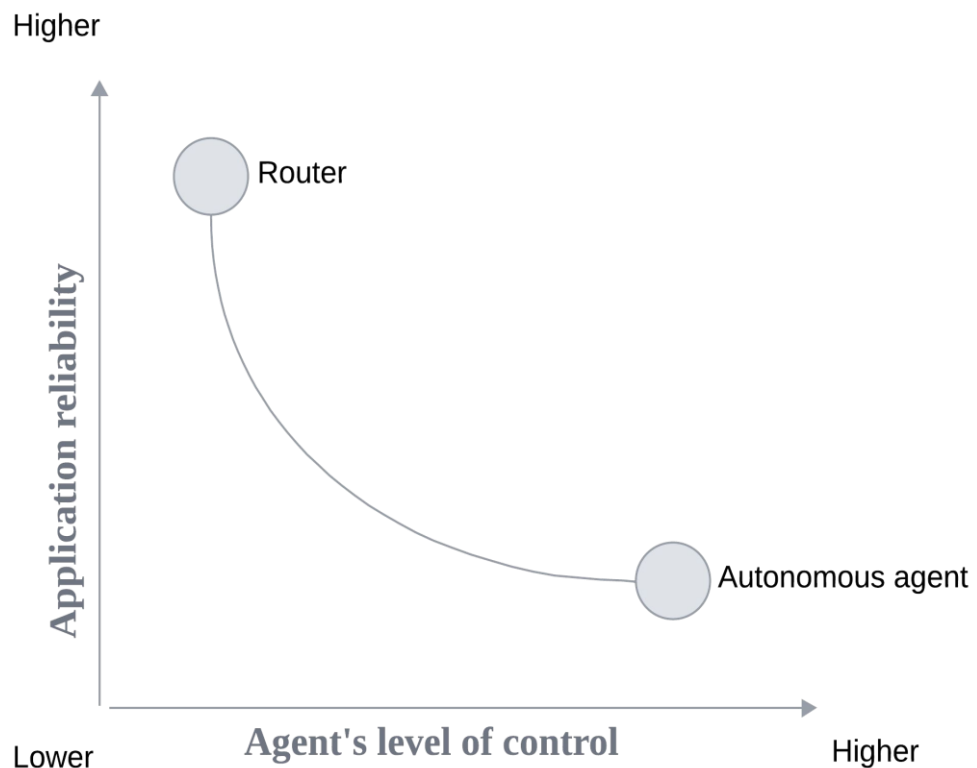
Less

More

Control

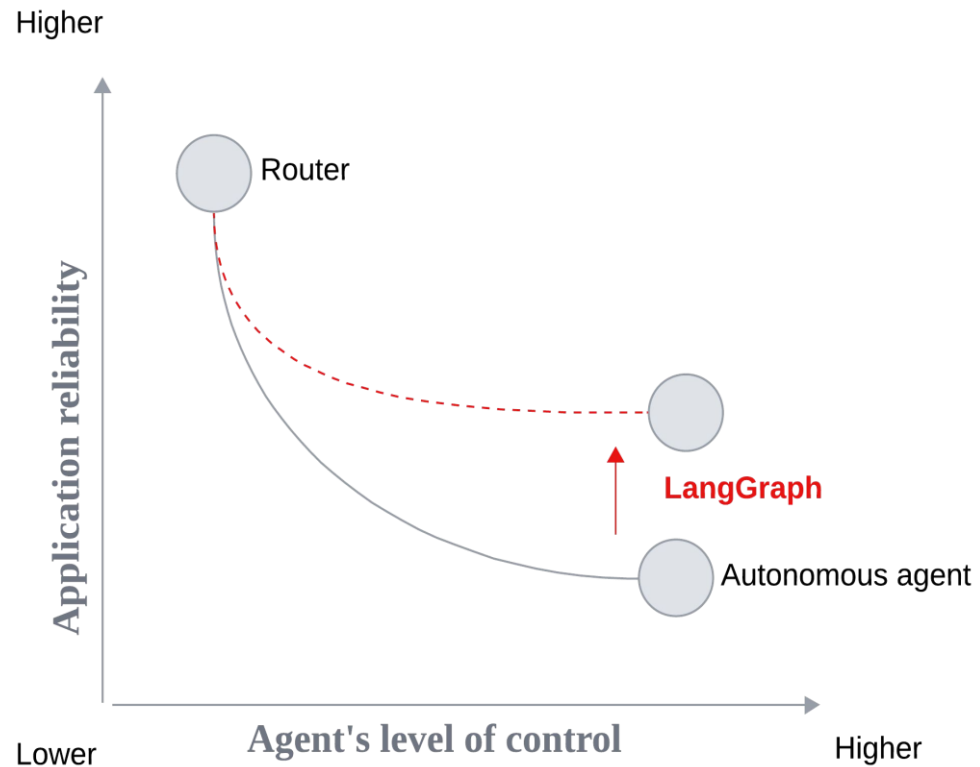


But, practical challenges.





LangGraph helps you bend the reliability curve

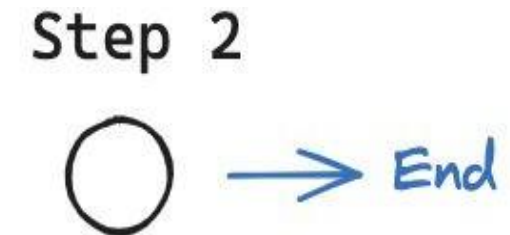
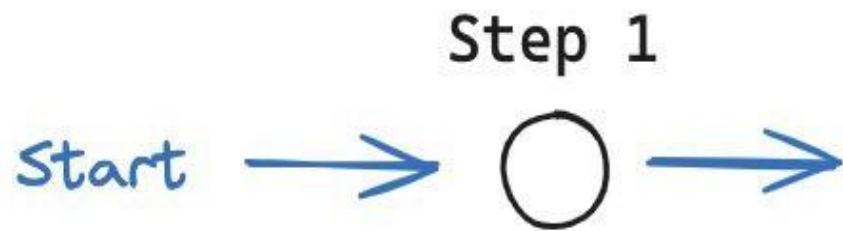




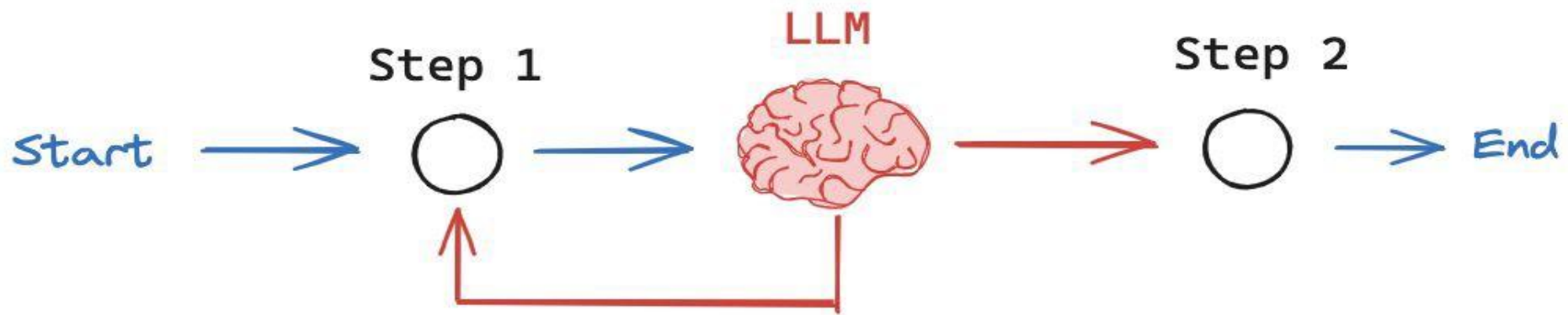
Balance reliability with control



Intuition: Let developer set parts of control flow (reliable)

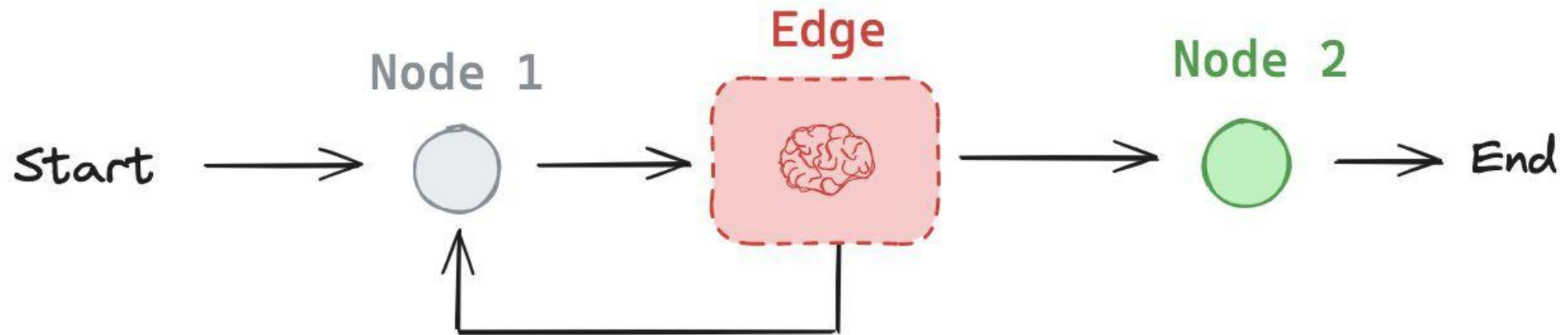


Intuition: Inject LLM to make it an agent (control)





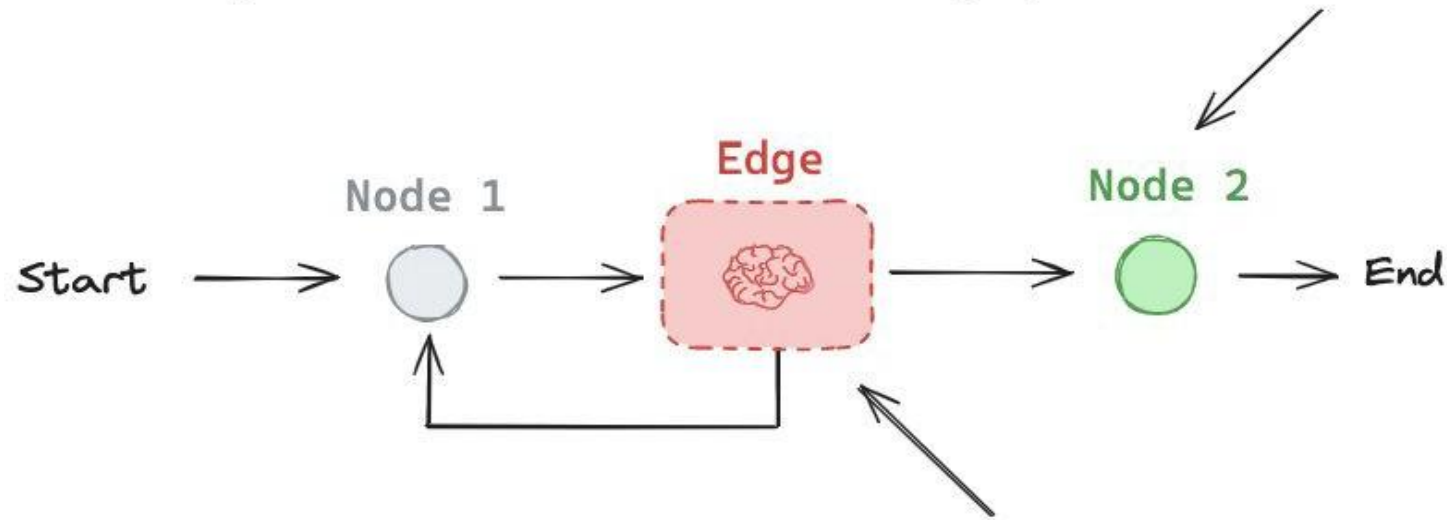
 LangGraph Express custom control flows as graphs





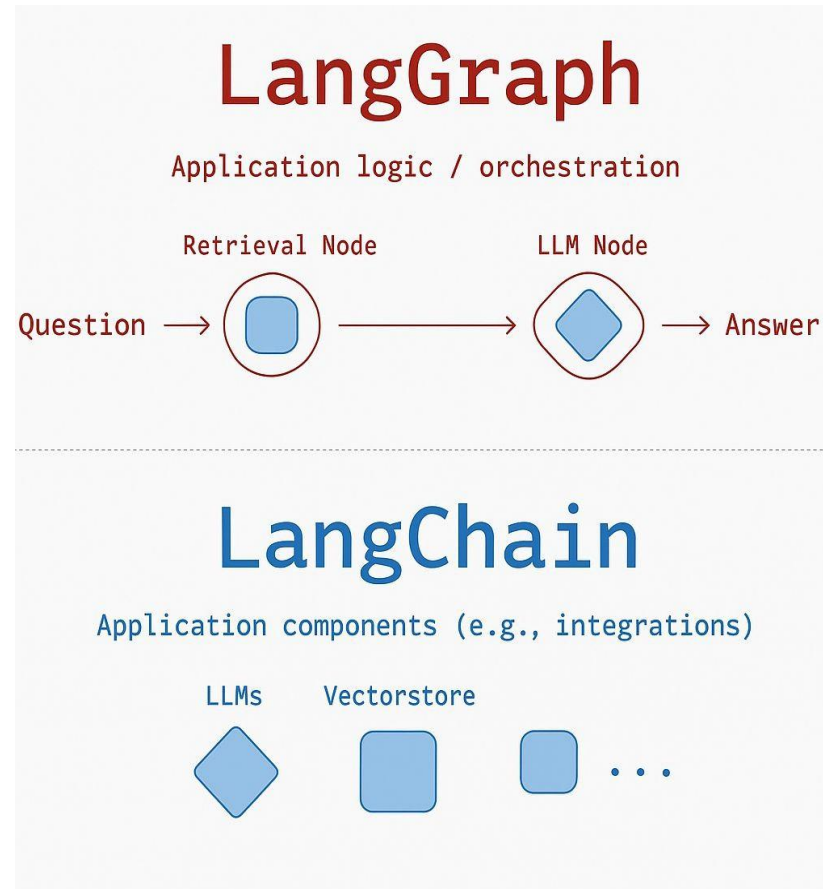
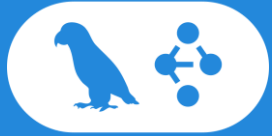
Memory: Shared state across the graph

Tools: Nodes can call tools and modify state

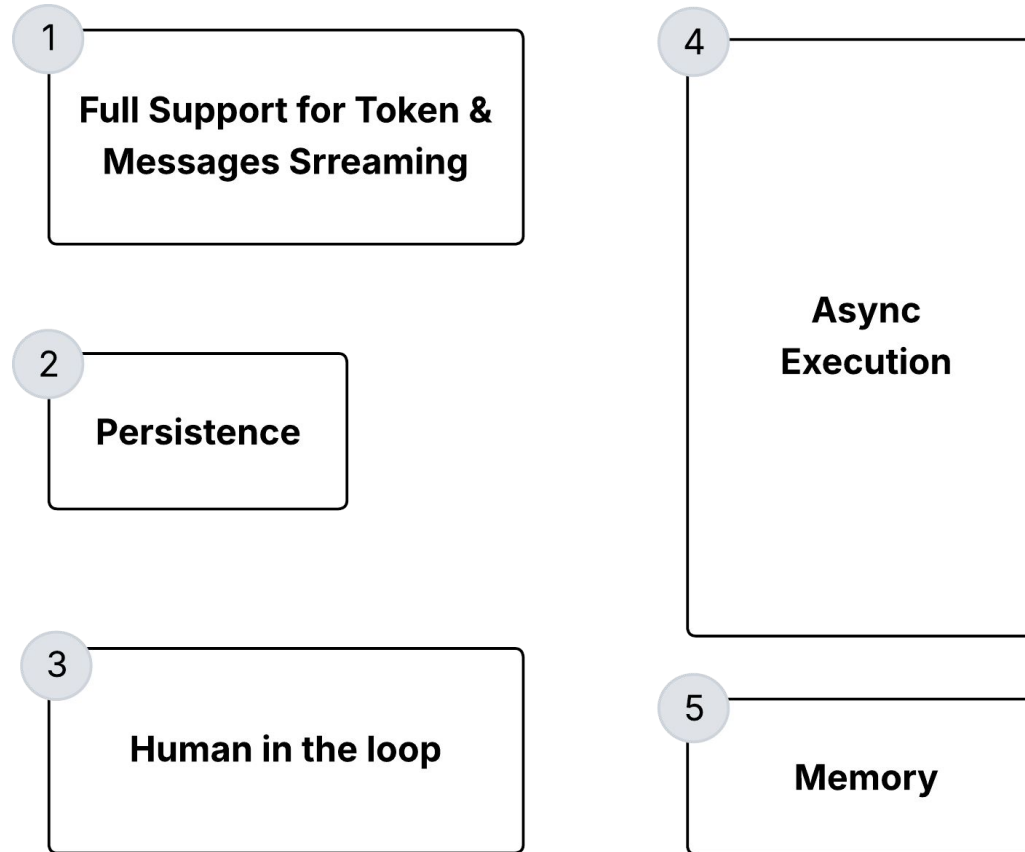
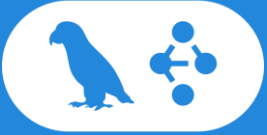


Planning: Edges can route control flow based on LLM decisions

LangChain vs LangGraph



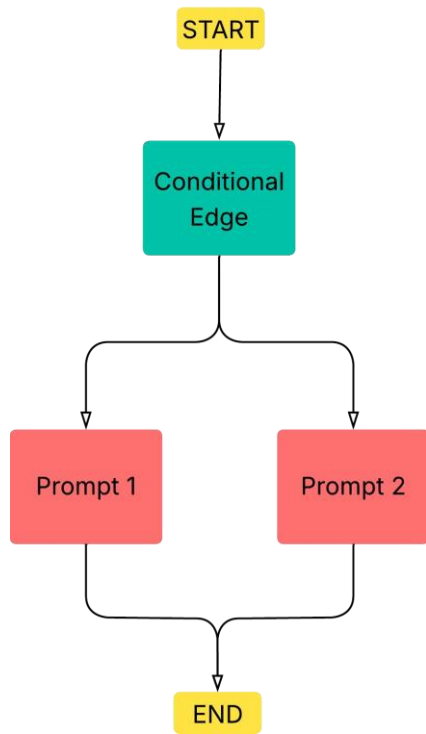
LangGraph core capabilities



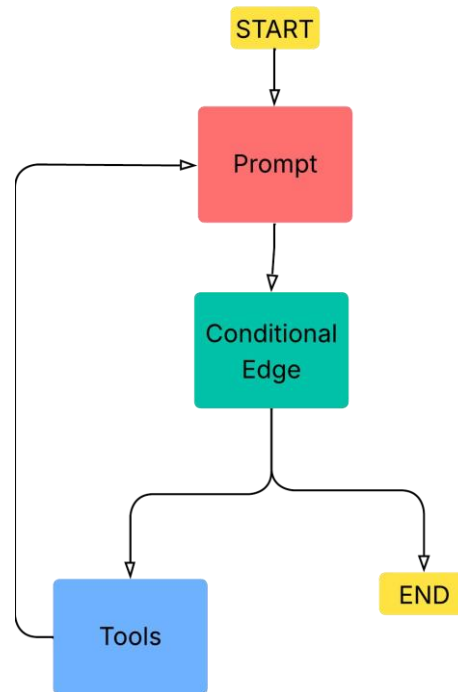
Agent Architectures



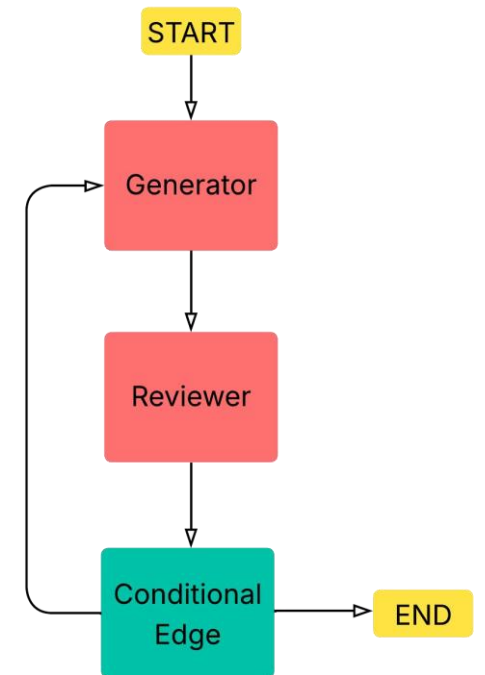
Router



React Agent



Reflection Agent

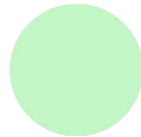


LangGraph building Blocks



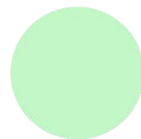
Code to Execute

NODE



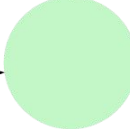
Node to run next

NODE 1



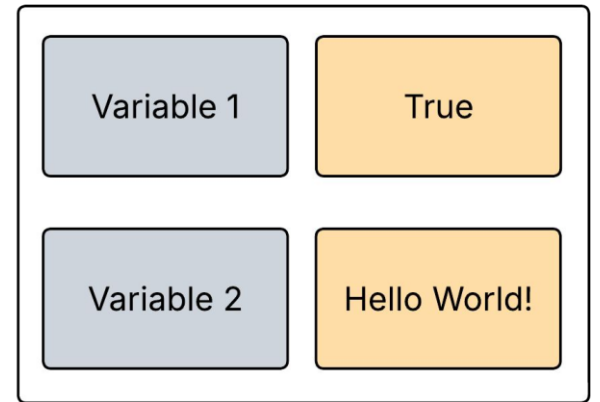
EDGE

NODE 2



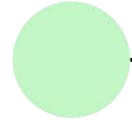
Stores inputs, outputs and
variables
Accessible everywhere

Graph State



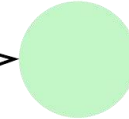
Code to execute &
decide which code to go to
next

NODE 1

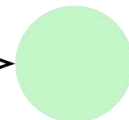


Conditional
Edge

NODE 2



NODE 3



Elements

LangGraph Building Blocks



- The **State** is a shared data structure that holds the current information or context of the entire application.
- In simple terms, it is like the application's memory, keeping track of the variables and data that nodes can access and modify as they execute.

Analogy:

Whiteboard in a Meeting Room: Participants (nodes) write and read information on the whiteboard (state) to stay updated and coordinate actions.





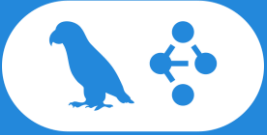
- **Nodes** are individual functions or operations that perform specific tasks within the graph.
- Each node receives input (often the current state), processes it, and produces an output or an updated state.

Analogy:

Assembly Line Stations: Each station does one Job - attach a part, paint it, inspect quality, and so on.



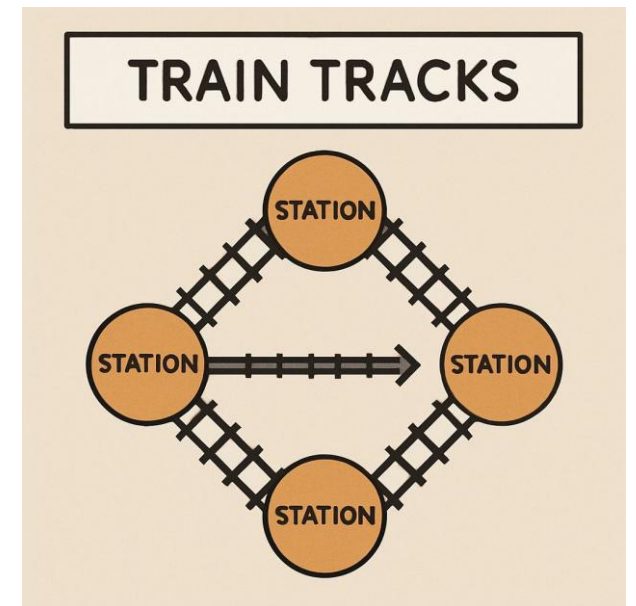
Edges



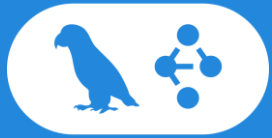
- **Edges** are the connections between nodes that determine the flow of execution.
- They tell us which node should be executed next after the current one completes its task.

Analogy:

Train Tracks: Each track (edge) connects the stations (nodes) together in a specific direction.



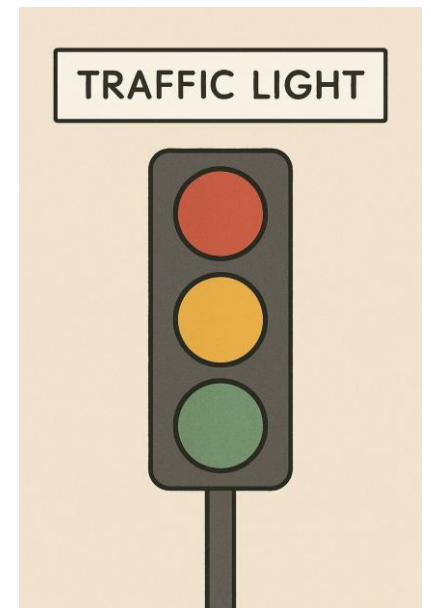
Conditional Edges

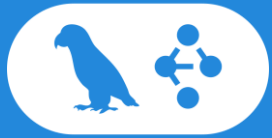


- **Conditional Edges** are specialized connections that decide the next node to execute based on specific conditions or logic applied to the current state.

Analogy:

Traffic Lights: Green means go one way, red means stop, yellow means slow down. The condition (light color) decides the next step.

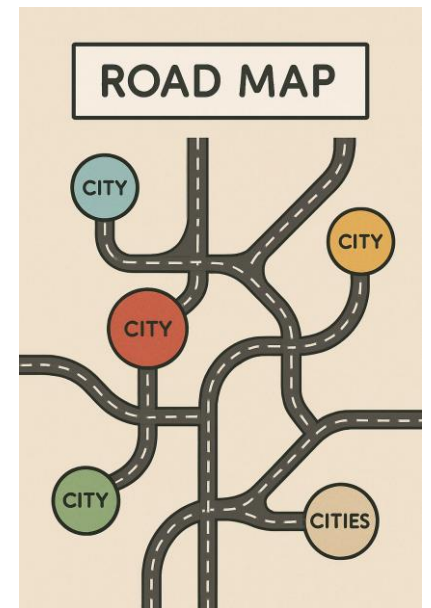




- A **Graph** in LangGraph is the overarching structure that maps out how different tasks (nodes) are connected and executed.
- It visually represents the workflow, showing the sequence and conditional paths between various operations.

Analogy:

Road Map: A road map displaying the different routes connecting cities, with intersections offering choices on which path to take next.



START



- The **START** node is a virtual entry point in LangGraph, marking where the workflow begins.
- It doesn't perform any operations itself but serves as the designated starting position for the graph's execution.

Analogy:

Race Starting Line: The place where a race officially begins.



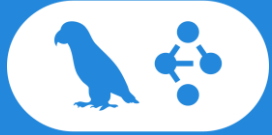


- The **END** node signifies the conclusion of the workflow in LangGraph.
- Upon reaching this node, the graph's execution stops, indicating that all intended processes have been completed.

Analogy:

Finish Line in a Race: The race is over when you cross it.

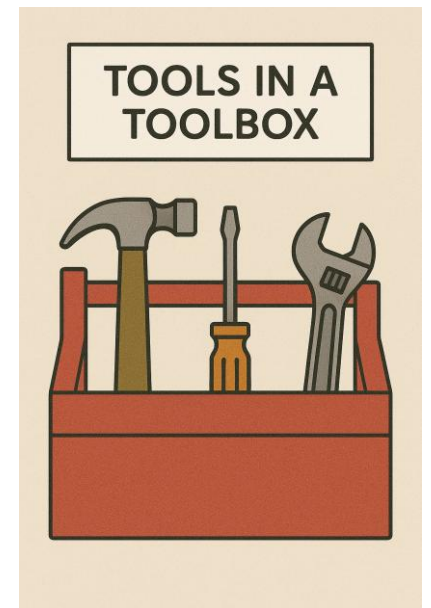


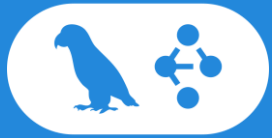


- **Tools** are specialized functions or utilities that nodes can utilize to perform specific tasks such as fetching data from an API.
- They enhance the capabilities of nodes by providing additional functionalities.
- Nodes are part of the graph structure, while tools are functionalities used within nodes

Analogy:

Tools in a Toolbox: A hammer for nails, a screwdriver for screws, each tool has a distinct purpose.



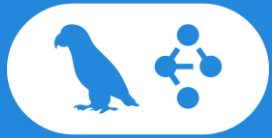


- A **ToolNode** is just a special kind of node whose main job is to run a tool.
- It connects the tool's output back into the State, so other nodes can use that information.

Analogy:

Operator Using a Machine: The operator (ToolNode) controls the machine (Tool), then takes the results back to the assembly line.

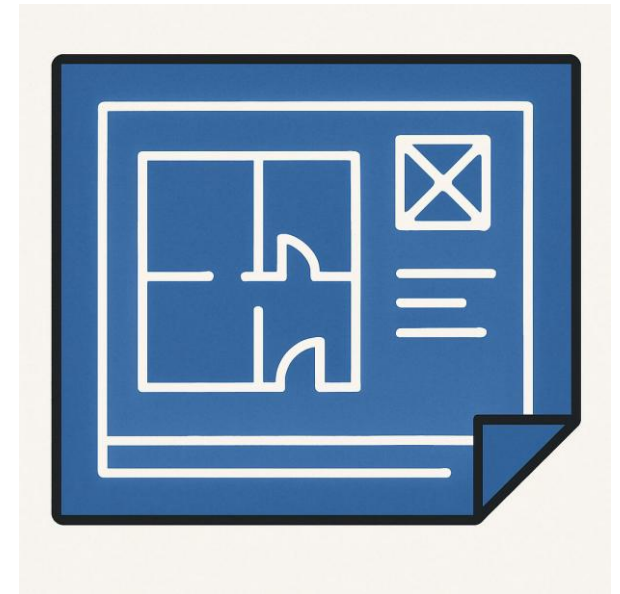




- A **StateGraph** is a class in LangGraph used to build and compile the graph structure.
- It manages the nodes, edges, and the overall state, ensuring that the workflow operates in a unified way and that data flows correctly between components.

Analogy:

Blueprint of a Building: Just as a blueprint outlines the design and connections within a building, a StateGraph defines the structure and flow of the workflow.

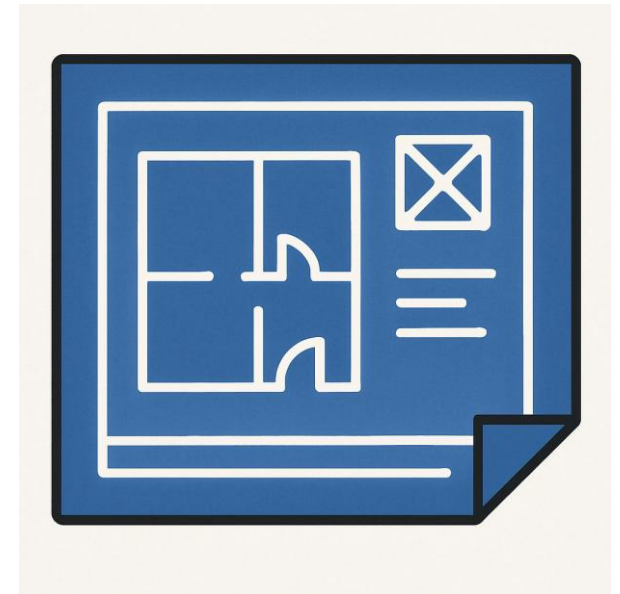




- A **Runnable** in LangGraph is a standardized, executable component that performs a specific task within an AI workflow.
- It serves as a fundamental building block, allowing for us to create modular systems.

Analogy:

LEGO Brick: Just as LEGO bricks can be snapped together to build complex structures, Runnables can be combined to create sophisticated AI workflows.



Messages



- **Human Message:** Represents input from a user.
- **System Message:** Used to provide instructions or context to the model
- **Function Message:** Represents the result of a function call
- **AI Message:** Represents responses generated by AI models
- **Tool Message:** Similar to Function Message, but specific to tool usage

Thank You